WIZARD'S TRIAL

A complete 3DGEngine game tutorial using every engine system

C++ AND LUA EDITION

Build an animated wizard, author staff-tip fireballs, create grounded behavior-tree enemies, connect HUD and audio, and ship a cooked game.

Project workflow | Native assets | Animation | Physics | AI | Particles | Audio | UI | Cameras | Packaging

A complete 3DGEngine game tutorial using every engine system

Build a third-person wizard action game from an empty project to a packaged standalone executable. The player explores a ruined observatory, crosses an outdoor landscape, solves a physics gate, fights animated wizard enemies, casts socket-based fireballs, and defeats an arena guardian. The game includes a title screen, opening cinematic, checkpoints, adaptive music, HUD, enemy health bars, navigation, behavior trees, particles, audio, post-processing, and a victory sequence. The observatory, landscape, dungeon, and arena are authored as level assets in one streamed world. Combat uses named timers, hit stops, and global time dilation for responsive spell impacts and victory slow motion.

Gameplay scripts in this edition can be authored in either native C++ or Lua. The two languages use the same scene objects, fields, animation events, sockets, audio, particles, cameras, game state, and packaged Content directory. Use Lua for fast iteration, C++ for native extensions, and mix both languages per object.

This tutorial is organized as production milestones. Finish the completion test at the end of each milestone before moving on.

1. What you will build

The final game, **Wizard's Trial**, has this flow:

1. A title HUD starts the game.
2. An opening camera sequence flies over an outdoor magical landscape.
3. The player takes control of an animated wizard carrying a socketed staff.
4. A rune gate uses triggers, physics joints, lights, audio, and particles.
5. Enemy wizards patrol on a baked navigation mesh.
6. Perception and a behavior tree make enemies focus, chase, cast, retreat, and clear focus when the player is lost.
1. Fireballs spawn from staff-tip sockets and damage only after a confirmed projectile collision.
1. Full-body actions lock movement; masked casting can preserve locomotion.
2. The player HUD displays health, score, elapsed time, spell state, and game state. Enemies display world-space health bars.
1. Dead enemies enter ragdoll and add score.
2. The arena victory triggers a camera sequence, music state, particles, save data, a short slow-motion beat, and a results HUD.
1. Level-as-asset streaming keeps the persistent game manager and player resident while loading the landscape, dungeon, and arena around them.
1. The project is validated, cooked, and launched through the standalone player.

2. Complete system coverage

Engine system	How Wizard's Trial uses it
Core application, window, timing	Editor Play and player use variable rendering, fixed simulation, scaled gameplay time, and an unscaled hit-stop clock
ECS, scenes, worlds, and streaming	Every actor is an entity with components; a persistent level and placed scene assets form a streamed `.3dgworld`
Prefabs and empty objects	Reusable fireballs, impacts, enemies, pickups, managers, and trigger volumes
Native assets and registry	Imported meshes, skeletons, animations, textures, materials, shaders, audio, and dependencies
Rendering	Static and skinned meshes, PBR, clustered lights, culling, framebuffers, and GPU profiling
Lighting and shadows	Directional, point, spot, area, cascaded, point, and spot shadows
Environment	Procedural sky, day/night, clouds, IBL, terrain, toggleable randomized grass, spline-driven water, and indoor occlusion
Post-processing	Bloom, SSAO, SSR, FXAA/MSAA, render scale, and a custom post-process shader
Materials and shader graphs	Stone, metal, water-side rock, emissive runes, dissolve shield, and HUD image shader
Animation	Skeletal import, locomotion blend space, jump states, transitions, action clips, events, root motion, and sockets
Physics	Rigid bodies, all collider families, channels, triggers, contacts, queries, joints, controller, stairs, and ragdoll
AI and navigation	Nav bounds, navmesh, grid fallback, A*, steering, perception, teams, state machine, BT, blackboard, tasks, decorators, services
Gameplay	Player controller, health, projectiles, attachments, game mode, C++/Lua scripts, runtime spline manipulation, named timers, global time dilation, and hit stops
Cameras	Third-person, first-person preview, isometric zone, collision, shake, saved cameras, editable splines, and sequences
Audio	Engine audio assets, spatial sources, cues, buses, effects, snapshots, adaptive music, occlusion, and doppler
Particles	CPU/GPU simulation, spawn/update/render modules, fireball, trail, impact, weather dust, and scripted control
UI and HUD	Title, pause, gameplay, health, mana-style cooldown, score, time, game state, buttons, images, and world health bars
Editor	Content browser, viewport, hierarchy, inspector, every specialist editor, console, profiler, and debug panels
Runtime and testing	Script compiler, validation, asset cooking, exported scenes, automated tests, and standalone player

The source-coverage checklist at the end of the tutorial provides a final signoff for every public subsystem.

3. Milestone roadmap

Milestone	Deliverable	Completion test
M0	Project and Content layout	The project reopens its last saved scene
M1	Imported engine-owned assets	Assets show native types and valid registry dependencies
M2	Streamed world, landscape, and observatory	Persistent and streamed levels load correctly; terrain, water, grass, sky, materials, and shadows render
M3	Physics graybox	Player, camera, stairs, gate, triggers, and collision channels behave correctly
M4	Animated player character	Idle, walk, run, jump, direction changes, and camera are smooth
M5	Staff, sockets, and action clips	Staff follows the hand; socket gizmo matches the visible staff tip
M6	Fireball effects and audio	Fireball, trail, impact, whoosh, and impact cue preview correctly
M7	Player spell scripting	A single input produces one timed projectile from the staff tip
M8	Rules, checkpoint, and scenes	Score, time, save data, restart, and scene loading work
M9	Enemy character and navigation	Enemy remains grounded and follows valid paths
M10	Behavior tree combat	Enemy patrols, sees, focuses, chases, casts, retreats, and clears focus
M11	Damage, death, and ragdoll	Projectile damage occurs only on collision and death activates once
M12	HUD and menus	All bound values and buttons work in Play
M13	Cameras and cinematic	Opening, arena reveal, shake, zones, and victory camera work
M14	Polish and performance	Audio mix, post-process, particles, debug, and profiler are clean
M15	Release	Validation passes and the packaged player launches

4. M0 - Create the project

4.1 Build and open the tools

From the repository root:

POWERSHELL

```
cmake -S . -B build
cmake --build build --config Debug --target 3DGEditor player
```

Open `3DGEditor`, create a project named `WizardsTrial`, and save its first scene as:

TEXT

```
Content/Scenes/Observatory.scene
```

4.2 Create the Content structure

Use the Assets panel rather than Windows Explorer:

TEXT

```
Content/
  Assets/
    AI/
    AnimationClips/
    AnimationGraphs/
    Audio/
    Characters/
    HUD/
    Materials/
    Meshes/
    Particles/
    Prefabs/
    Shaders/
    Textures/
    Worlds/
  Scenes/
  Scripts/
```

The editor stores project scripts in `Content/Scripts`. C++ `.h` scripts compile into the shared game module; Lua `.lua` scripts load directly in Editor Play and the standalone player. In the Inspector or Character Editor, choose the script language, enter a class or file name, select a template, then click **Create Script** and **Attach Selected**. Use **Detect Fields** after adding a Lua `Fields` table. Lua files appear in the saved-script dropdown and open by double-clicking them in the Content browser.

4.3 Establish stable names

Use these names throughout:

TEXT

```
PlayerWizard
PlayerStart
GameManager
Staff
FireballPrototype
FireballImpactPrototype
EnemyWizard_01
ArenaGuardian
NavMeshBoundsVolume_Main
RuneGate
ArenaTrigger
WizardsTrialWorld
LandscapeLevel
DungeonLevel
ArenaLevel
```

Names are used by small-game lookups, script fields, triggers, camera sequences, and diagnostics.

Completion test

- Save the scene.

- Close and reopen the editor.
- Confirm `Observatory.scene` is restored rather than a generated scene.
- Confirm the Console reports the project Content root.

5. M1 - Import engine-owned assets

5.1 Import through the Content browser

Click **Import Asset**, browse the operating-system folders, choose the source, and then choose the destination folder under Content. Do not manually type an external path into a runtime field.

Import:

- an animated wizard FBX;
- a static staff mesh;
- observatory architecture;
- idle, walk, run, jump-start, jump-loop, jump-land, cast, hit, and death clips;
- albedo, normal, ORM, height, emissive, and UI images;
- whoosh, impact, footsteps, ambience, enemy cast, music stems, and dialogue.

5.2 Static mesh import

In Static Mesh Import Settings:

1. Choose scale and axis conversion.
2. Generate normals/tangents only if the source lacks them.
3. Enable material-slot extraction where useful.
4. Choose collision generation only for simple props.
5. Save the result as an engine-owned `.3dgmesh`.

5.3 Skeletal import

In Skeletal Import Settings:

1. Import the wizard model in **Skeletal Mesh** mode. Choose **Create from

source **and keep** Import Skeleton Asset** enabled for the first character. For another mesh using the same rig, choose the existing wizard skeleton and leave skeleton import disabled to avoid duplicating it.

1. Verify bone count and hierarchy.
2. For each separate animation source, choose **Import As > Animation**.
3. Choose the wizard's existing `.3dgskel` from the **Skeleton** dropdown.
4. Leave **Import Mesh Asset** and **Import Skeleton Asset** off. This writes

only `.3dganim` clips and avoids duplicating the wizard mesh and rig for every animation FBX.

1. Give every clip a unique readable name such as `Idle`, `WalkForward`,

`RunForward`, `JumpStart`, `JumpLoop`, `JumpLand`, `StaffCast`, `HitReact`, and `Death`.

1. Save engine-owned skeletal and animation data.

Do not rely on repeated source names such as `Unreal Take`. The animation graph, action clips, scripts, and diagnostics all become clearer when clip names are unique.

5.4 Texture import

Assign correct color space:

- albedo and emissive color: sRGB;

- normal, ORM, height, masks, and data: linear;
- use 16-bit PNG decoding for height or high-precision grayscale sources.

The native texture keeps import metadata and runtime payload under Content.

5.5 Asset registry and reimport

Open the registry diagnostics and confirm every native asset has:

- a stable asset handle;
- a project-relative path;
- its asset type;
- source import metadata;
- registered dependencies.

Use **Reimport** to update an asset without breaking references. Reimport should replace the native payload while preserving its identity.

Completion test

- Double-click each imported asset and confirm the appropriate editor opens.
- Confirm no material or model warns about missing registry dependencies.
- Move one source file outside the project and confirm the native asset still

previews.

6. M2 - Build the world and exercise the renderer

6.1 Terrain, grass, and water

Add or import a landscape heightmap, including a Gaea export if available. Configure:

- terrain scale and height;
- layer textures for rock, soil, and moss;
- grass density and base height;
- **Randomize Grass Height** with a minimum and maximum multiplier, such as

`0.75x` to `1.25x`, to break up repeated silhouettes;

- a water plane for the observatory pool;
- reflection/refraction and shoreline placement.

The randomizer changes vertical height only. Disable it when the design needs a uniform lawn or readable gameplay boundary. With the toggle off, every blade uses the authored **Grass Height**. With it on, each blade receives a stable height multiplier inside **Height Range**; saving and reopening the scene keeps the toggle and range. Changing the range rebuilds the grass scatter without rebuilding the terrain.

For natural results, start with a restrained range:

Grass use	Base height	Random height range
Trimmed observatory lawn	0.35	Off
Outdoor meadow	0.60	0.75x - 1.25x
Magical overgrowth	0.85	0.55x - 1.45x

Paint grass only where required. High density across a large 1024-resolution terrain can dominate CPU scatter time and GPU overdraw. Validate density using the Profiler before filling the whole landscape.

6.1.1 Create a spline-driven river

Add a river water object and its river spline. Select the water object, assign the spline as its flow spline, then edit the control points in the viewport. The river mesh follows the spline's position, smooth bends, point rotations, width, and flow direction. Move the water object to move the river as one unit; select an individual spline point when shaping a bend or banking a section.

Use enough control points to describe the turn without placing several points almost on top of each other. Very sharp reversals produce pinched banks and should be replaced by a wider curve.

Scripts can animate or procedurally reshape a spline during Play. Runtime changes update the connected river immediately and reset when Play ends, so the saved editor level is not modified.

Native C++ uses zero-based point indices:

CPP

```cpp
class RiverPulse final : public engine::gameplay::Script {
public:
    void OnStart() override {
        river = FindObject("ObservatoryRiverSpline");
    }

    void OnUpdate(float dt) override {
        time += dt;
        glm::vec3 point = GetSplinePoint(river, 1);
        point.y = baseY + std::sin(time * 0.5f) * 0.15f;
        SetSplinePoint(river, 1, point);
    }

private:
    engine::ecs::Entity river = engine::ecs::kNull;
    float time = 0.0f;
    float baseY = 0.2f;
};
```

Lua uses one-based point indices:

LUA

```lua
local river = nil
local time = 0.0
local baseY = 0.2

function OnStart()
    river = Engine.FindObject("ObservatoryRiverSpline")
end

function OnUpdate(dt)
    time = time + dt
    local x, y, z = Engine.GetSplinePoint(river, 2)
    Engine.SetSplinePoint(river, 2, x, baseY + math.sin(time * 0.5) * 0.15, z)
end
```

Other available spline operations include adding, inserting, and removing points; setting point rotations; translating the whole spline; toggling a closed loop; and sampling a position or tangent from normalized distance `0..1`. Use `SplineTangentAt` to orient a moving platform, camera target, boat, or effect along the path.

Keep the arena and dungeon floors as authored static meshes where exact collision is important. Use terrain for the outdoor approach.

6.2 PBR materials

In Material Maker create:

Material	Key properties
`M_Stone`	albedo, normal, ORM, height, roughness 0.8
`M_Bronze`	metallic 0.9, roughness 0.3, normal
`M_RuneGlow`	emissive blue/orange, bloom-visible strength
`M_WizardCloth`	rough cloth, sheen, normal
`M_Crystal`	transparent blend, transmission, IOR, thickness
`M_WetRock`	lower roughness, stronger normal, dark albedo

Use engine texture dropdowns. Save material assets and apply them to imported static and skeletal material slots. If a mesh appears black, check the asset registry dependencies, decoded textures, shader validity, and light exposure.

6.3 Shader graphs

Create three shader assets:

1. **RunePulse** - Surface domain; multiply rune color by a time-driven pulse and feed Emissive.
1. **ShieldDissolve** - Surface domain; compare a noise value with a scalar

dissolve parameter, drive opacity/mask, and add a bright edge.

1 **CinematicVignette** - Post-process domain; darken screen edges during the boss reveal.

Drag from a typed pin to empty graph space to use the compatible-node prompt. Right-click a node to delete it. Give every node a unique label/ID. Compile, save, and apply the correct shader domain.

6.4 Lighting

Add:

- one Directional Light for the sun/moon;
- Point Lights in rune lanterns;
- Spot Lights above arena entrances;
- a soft Area Light in the indoor observatory.

Enable the corresponding shadow types. Tune cascaded directional shadow distance in World Settings so shadows remain visible across the arena. Build or refresh lighting/occlusion after enclosing a room.

6.5 Sky, clouds, and IBL

In World Settings:

- enable procedural sky and choose a late-afternoon time;
- enable clouds and cloud shadows;
- set wind direction and speed;
- enable IBL;
- enable Skylight Occlusion for indoor darkness;
- keep a nonzero minimum indoor skylight so black surfaces retain detail.

Cloud rendering and cloud shadow toggles are independent. If cloud settings are off, cloud artifacts must not affect the scene.

6.6 Post-processing and anti-aliasing

Enable and tune:

- bloom for runes and spells;
- SSAO for contact and indoor depth;
- SSR around the pool and polished metal;
- FXAA for the post-process path;
- MSAA for direct view if required;
- render scale at `1.0` while validating image quality.

Add `CinematicVignette` to the post-process shader stack only during the reveal sequence.

6.7 Renderer diagnostics

Use the Profiler to distinguish:

- scene submission;
- CPU renderer time;

- GPU scene time;
- GPU UI time;
- total GPU frame time.

Frustum culling should exclude distant landscape props. Clustered lighting keeps the many rune lights practical.

6.8 Compose the streamed world

Save the game as separate level assets:

TEXT

```
Content/Scenes/Observatory.scene    persistent level
Content/Scenes/Landscape.scene      distance streamed
Content/Scenes/Dungeon.scene        distance or manual
Content/Scenes/Arena.scene          distance or manual
```

Open **Panels > World Editor**, click **New World**, and save:

TEXT

```
Content/Assets/Worlds/WizardsTrialWorld.3dgworld
```

Set `Observatory.scene` as the persistent level. Add the other three scenes as streamed levels and place their world transforms. Use separate load and unload radii so the player cannot oscillate across one boundary and repeatedly reload a level. A practical starting point is:

Level	Rule	Load radius	Unload radius
Landscape	Always Loaded	-	-
Dungeon	Distance	55	75
Arena	Manual	-	-

Use **Cook World** to export every referenced editor scene, calculate bounds, and create a runnable world manifest. Double-clicking the `.3dgworld` asset reopens World Editor.

The persistent level should contain `PlayerWizard`, `GameManager`, global HUD selection, persistent music state, and any object that must survive level activation. A streamed level owns its local geometry, colliders, navigation bounds, lights, enemies, particles, and audio sources. When it unloads, its scripts receive `OnDestroy` before its entities are removed.

The player runtime loads or unloads at most one level per streaming update. Shared native assets remain cached, reducing repeated GPU uploads.

Completion test

- Static and skinned objects respond to PBR lighting.
- Directional, point, and spot shadows have correct shape and range.
- Indoor rooms are darker than outdoors.
- Water reflects the environment without covering opaque geometry incorrectly.
- Grass height is uniform when its randomizer is off and remains inside the

configured multiplier range when it is on.

- Editing or scripting the river spline updates the river mesh and flow without

breaking at bends.

- Shader graph materials compile without ImGui ID warnings.
- GPU and CPU frame times are recorded for later comparison.
- The persistent observatory remains resident while distance levels activate

and unload without duplicate suns, scripts, physics bodies, or audio voices.

7. M3 - Build collision and the physics puzzle

7.1 Collider coverage

Use the matching shape where practical:

- box for walls and gates;
- sphere for rune pickups;
- capsule for characters;
- cylinder for columns;
- cone for crystal spikes;
- plane for simple floors;
- torus/pyramid/stair colliders for their matching primitives when used.

Enable line-based collider guides. Guides are editor/debug lines, not cube meshes, and can be disabled in Play.

7.2 Collision channels and presets

Create or use these channels:

TEXT

```
WorldStatic
WorldDynamic
Player
Enemy
Collectible
Projectile
CameraBlocker
Trigger
```

Recommended responses:

Object	Player	Enemy	Projectile	Camera
WorldStatic	Block	Block	Block	Block
Collectible	Overlap	Ignore	Ignore	Ignore
Player	-	Block	enemy projectile overlap/block	Ignore
Enemy	Block	Block	player projectile overlap/block	Ignore
Trigger	Overlap	Overlap	Ignore	Ignore

The projectile owner is also ignored by runtime projectile collision.

7.3 Character controller and stairs

Start with:

TEXT

```
Capsule radius       0.40
Capsule height       1.80
Step height          0.35
Maximum slope        50 degrees
Ground probe         0.25
Gravity              -9.81
Maximum fall speed   35
```

Test capsule movement up the staircase. Step smoothing should prevent the character and camera from snapping at every riser.

7.4 Physics gate

Create `RuneGate` as a dynamic rigid body. Connect it to a frame with a distance, rope, or spring joint. Add a trigger named `GateRuneTrigger`.

The trigger does not push the player. A script or trigger action releases or moves the gate after the rune is collected. Use a Mover component for a simple authored gate, or a joint for a physically swinging gate.

7.5 Queries and debug

Use ray or shape queries for:

- ground probing;
- camera collision;
- AI line of sight;
- spell collision;
- interaction targeting.

Enable physics debug only while diagnosing. Verify contact normals and collider boundaries, then turn them off for final Play.

Completion test

- The player cannot pass through walls.
- Pickups never stop the player or retract the camera.
- Stairs are smooth.
- The gate moves without tunneling or exploding.
- Projectiles stop at the first wall and never damage a target behind it.

8. M4 - Create the animated player character

8.1 Character asset

Open Character Editor and create:

TEXT

```
Content/Assets/Characters/PlayerWizard.3dgcharacter
```

Configure:

- skeletal model and material assets from searchable dropdowns;
- render-only orientation, position offset, and scale;
- capsule collision;
- walk, run, jump, slope, and step settings;
- Health `100 / 100`;
- gameplay team `1`;
- third-person camera settings;
- scripts and later the animation graph.

Render-only model correction must not change the character's scene transform or gizmo axes. Scene placement uses the ordinary scene transform.

8.2 Camera policy

Open Game Mode Settings and select the default gameplay camera mode before Play. Do not bind runtime input to toggle camera modes. Use:

- third-person for normal play;
- an isometric camera zone for the rune puzzle;
- saved cameras and sequences for cinematics.

Set camera collision radius, wall padding, retract speed, return speed, target height, distance, yaw, and pitch.

8.3 Place the player

Add the character asset to the scene and name it `PlayerWizard`. Place it above the floor so the capsule can settle. Add or keep a `PlayerStart` marker if the game-mode workflow uses one.

Completion test

- The model faces gameplay forward.
- Move, scale, and rotation gizmos follow scene axes.
- The capsule encloses the visible character.
- Camera collision retracts and returns smoothly.

9. M5 - Author locomotion and action animation

9.1 Animation graph

Create:

TEXT

```
Content/Assets/AnimationGraphs/PlayerWizard.3dggraph
```

Double-clicking the asset should open Graph Editor with the character asset as its preview model.

Add parameters:

Parameter	Type	Purpose
`Speed`	Float	idle/walk/run blend
`Direction`	Float	strafe/turn direction
`VerticalSpeed`	Float	rising/falling
`Grounded`	Bool	floor state
`Jump`	Trigger	jump start
`Land`	Trigger	landing

9.2 Locomotion blend space

Create a 1D or 2D Locomotion state:

- idle at Speed `0`;
- walk at approximately `2.5`;
- run at approximately `6.0`;
- optional left/right movement samples on the Direction axis.

Enable synchronized samples and tune parameter damping. Smooth character rotation separately with turn speed so direction changes do not snap.

9.3 Jump states and multi-condition transitions

Create `JumpStart`, `InAir`, and `Land`.

Example transitions:

TEXT

```
Locomotion -> JumpStart: Jump trigger
JumpStart  -> InAir: exit time AND Grounded == false
InAir      -> Land: Grounded == true AND VerticalSpeed <= 0
Land       -> Locomotion: exit time
```

Use the transition field's additional conditions and choose whether all or any conditions must pass.

9.4 Action clips

Open Clip Editor and create:

TEXT

```
StaffCast.3dgclip
HitReact.3dgclip
Death.3dgclip
```

For `StaffCast`:

- action name: `StaffCast`;
- clip: the uniquely named cast animation;
- fade in/out: approximately `0.08 / 0.15`;
- event: `Cast.Release` at the frame where the spell leaves the staff;
- optional mask root: upper spine for a layered cast.

For a heavy attack or death, leave the mask empty. A non-layered action blocks movement until the clip completes, like a montage. A masked action can preserve locomotion.

Use root motion only on clips authored for it. Ordinary locomotion should remain controller-driven.

9.5 Attach the graph and clips

In Character Editor:

- assign the animation graph;
- add all standalone action clips;
- click Refresh if a newly saved clip does not appear;
- preview locomotion by driving Speed, Direction, Grounded, and VerticalSpeed;
- preview action events and movement locking.

Completion test

- Idle, walk, and run blend smoothly.
- Direction changes do not snap.
- Jump uses all three states.
- Full-body Death blocks movement.
- Masked StaffCast can preserve movement if desired.
- `Cast.Release` fires once at the visible release frame.

10. M6 - Add the staff, sockets, and attachments

10.1 Staff attachment

In Character Editor add the staff model as an attachment:

TEXT

```
Bone:        RightHand
Socket name: StaffGrip
Material:    M_Bronze or staff material
```

Adjust attachment position, rotation, and scale in the character preview.

10.2 Spell socket

Add a second named socket:

TEXT

```
Socket name: StaffTip
Parent/bone: RightHand or staff attachment basis
```

Select the staff/socket item in Character Editor. The gizmo must appear at the socket so it can be positioned visually at the staff tip. Preview idle, run, jump, and cast to verify that the point follows animation.

At runtime, animation updates before attachments and socket queries. This gives scripts the current-frame world transform.

Completion test

- StaffGrip remains in the hand for every clip.
- StaffTip remains at the visible tip.
- No socket uses an editor-only scene object as its runtime source.

11. M7 - Create fireball particles and audio

11.1 Fireball particle system

Open Particle Editor and start with the Fire preset. Save:

TEXT

```
Content/Assets/Particles/FireballCore.particle
```

Configure the module pipeline:

Spawn stage

- rate `180-300/s`;
- lifetime `0.18-0.45 s`;
- sphere shape radius `0.08-0.15`;
- low random velocity and backward trail spread;
- maximum particles appropriate for the effect.

Update stage

- light drag;
- color over life from hot yellow to orange to transparent red;
- size over life from medium to small;
- gentle turbulence/force;
- rotation.

Render stage

- billboard renderer;
- additive blend;
- soft flame texture;
- optional trail/ribbon module.

Use Auto simulation. It selects GPU compute when supported and CPU otherwise. Temporarily force CPU when diagnosing a GPU-only preview problem.

11.2 Impact and ambient systems

Create:

- `FireballImpact.particle`: one-shot sphere burst;
- `StaffMuzzle.particle`: short burst at cast release;
- `RuneIdle.particle`: looping orbit/sparks;
- `ObservatoryDust.particle`: low-rate world-space dust;
- `EnemyDeath.particle`: burst on death.

Use local space for StaffMuzzle and world space for FireballImpact.

11.3 Audio assets and cues

In Audio Editor import/author:

TEXT

```
FireballWhoosh
FireballImpact
WizardFootsteps
EnemyCast
RuneActivate
GateOpen
ArenaAmbience
WizardMusic
```

Create randomized cues for footsteps and impacts. Set cooldown and maximum instances so rapid events do not create an unbounded voice count.

11.4 Mixer and adaptive music

In Audio Mixer configure buses:

TEXT

```
Master, Music, SFX, Dialogue, UI, Ambient
```

Create snapshots:

TEXT

```
Default, Paused, Indoor, Cinematic
```

Add light reverb indoors, compression where needed, and dialogue ducking. Create adaptive music states:

TEXT

```
Explore -> Alert -> Combat -> Victory
```

Use BPM/beat synchronization and crossfades.

11.5 Spatial setup

World sources use attenuation, doppler, cones, priority, and periodic occlusion. Route ambience to Ambient and spell effects to SFX. The active gameplay camera provides the listener.

Completion test

- Every particle asset previews and saves.
- Placed particles appear in the scene and Play.
- Cue variations can be heard.
- Indoor snapshot and occlusion are audible but not excessive.
- Music changes state without an abrupt restart.

12. M8 - Build spell prototypes and prefabs

12.1 Fireball prototype

Add an empty object named `FireballPrototype`. Give it:

- a small sphere/static mesh core;
- emissive `M_RuneGlow` or a fireball material;
- a sphere collider on Projectile channel;
- Projectile data;
- FireballCore particle component;
- spatial looping travel audio if desired;
- optional Point Light.

The object is a prototype, not a visible starting projectile. Save it as:

TEXT

```
Content/Assets/Prefabs/Fireball.3dgprefab
```

12.2 Impact prototype

Create `FireballImpactPrototype` with:

- FireballImpact particle system;
- short point light;
- impact cue;
- one-shot cleanup script.

Save a prefab and keep a prototype available to `SpawnFromObject` if that is the project's chosen spawn workflow.

12.3 Other reusable prefabs

Create prefabs for:

- EnemyWizard;
- RunePickup;
- HealthPickup;
- checkpoint;
- gate trigger;
- arena entrance torch.

Prefab instances own scene transforms and may apply instance overrides while the source prefab keeps reusable component defaults.

Completion test

- Placing the prefab creates independent runtime particle/audio state.
- Editing a scene transform does not alter the source prefab.
- Runtime asset dependencies include meshes, material, shader, textures, audio,

and particle assets.

13. M9 - Create the player combat script

Select `PlayerWizard`, add a script slot, enter `WizardCombat`, choose a suitable template, and click **Create Script**. Add inspector fields:

TEXT

```
projectilePrototype  String  FireballPrototype
castAction           String  StaffCast
releaseEvent         String  Cast.Release
whooshCue             Asset  Content/Assets/Audio/FireballWhoosh...
cooldown              Float  0.65
projectileSpeed       Float  18
projectileDamage      Float  30
```

Use the generated header under `Content/Scripts`. The core logic is:

CPP

```
#pragma once
#include <engine/gameplay/Script.h>
#include <engine/gameplay/GameplayComponents.h>
#include <glm/glm.hpp>

class WizardCombat final : public engine::Script {
public:
    void OnCreate() override {
        BindTimerFunction("ResetCastCooldown", [this]() {
            canCast_ = true;
        });
    }

    void OnUpdate(float) override {
        if (Input().MousePressed(0) && canCast_
            && !Anim().IsActionPlaying()) {
            const std::string action =
                GetFieldString("castAction", "StaffCast");
            if (Anim().PlayActionClip(action)) {
                pendingCast_ = true;
                canCast_ = false;
                SetTimerByFunctionName(
                    "ResetCastCooldown",
                    GetFieldFloat("cooldown", 0.65f),
                    false);
            }
        }

        const std::string eventName =
            GetFieldString("releaseEvent", "Cast.Release");
        if (pendingCast_ && WasAnimationEvent(eventName)) {
            ReleaseSpell();
            pendingCast_ = false;
        }
    }

private:
    void ReleaseSpell() {
        glm::vec3 tip;
        if (!SocketPosition("StaffTip", &tip)) return;

        const std::string prototype =
            GetFieldString("projectilePrototype",
                           "FireballPrototype");
        const engine::ecs::Entity bolt =
            SpawnFromObject(prototype, tip);
        if (bolt == engine::ecs::kNull) return;

        const auto* selfTransform = Transform();
        if (!selfTransform) return;
        const glm::vec3 forward = glm::normalize(
            selfTransform->rotation * glm::vec3(0, 0, 1));

        if (auto* projectile = TryGet<engine::Projectile>(bolt)) {
            projectile->owner = Self();
            projectile->dir = forward;
            projectile->speed =
                GetFieldFloat("projectileSpeed", 18.0f);
            projectile->damage =
                GetFieldFloat("projectileDamage", 30.0f);
        }

        const std::string cue = GetFieldAsset("whooshCue");
        if (!cue.empty()) Audio().PlayCue(cue, true);
        Camera().Shake(0.20f, 0.12f, 24.0f);
    }

    bool canCast_ = true;
    bool pendingCast_ = false;
};
```

This design prevents spam in four ways:

- input uses a pressed edge, not held state;
- cooldown must expire;
- another action cannot start while one is playing;
- projectile spawn occurs once on the authored animation event.

C++ has no automatic reflection for member-function names, so `BindTimerFunction` registers the callable once in `OnCreate`. `SetTimerByFunctionName` then creates a one-shot or repeating timer by that registered name. `IsTimerActive("ResetCastCooldown")` and `ClearTimerByFunctionName("ResetCastCooldown")` are available for diagnostics and cancellation. These timers consume scaled gameplay time.

The projectile system performs a swept collision, ignores its owner, stops at the first solid hit, and applies damage only if that collider owns living Health. Impact particles and audio use the confirmed hit position.

Compile in the editor. Verify the class appears in saved-script dropdowns, attach it, enable it, and inspect errors in Script Debug/Console.

13.1 Lua version

Choose **Lua** in the script-language dropdown, create `WizardCombat.lua`, and attach it to `PlayerWizard`. This version uses the same Inspector fields and animation event as the C++ version:

LUA

```
local canCast = true
local pendingCast = false

function OnCreate()
    Engine.Log("WizardCombat ready")
end

function ResetCastCooldown()
    canCast = true
end

function OnUpdate(dt)
    if Engine.WasMouseButtonPressed(0)
        and canCast
        and not Engine.IsAnimationActionPlaying() then
        local action = Engine.GetFieldString("castAction", "StaffCast")
        if Engine.PlayActionClip(action) then
            pendingCast = true
            canCast = false
            Engine.SetTimerByFunctionName(
                "ResetCastCooldown",
                Engine.GetFieldFloat("cooldown", 0.65),
                false)
        end
    end

    local releaseEvent =
        Engine.GetFieldString("releaseEvent", "Cast.Release")
    if pendingCast and Engine.WasAnimationEvent(releaseEvent) then
        releaseSpell()
        pendingCast = false
    end
end

function releaseSpell()
    local x, y, z = Engine.SocketPosition("StaffTip")
    if x == nil then
        Engine.Log("StaffTip socket was not found")
        return
    end

    local prototype = Engine.GetFieldString(
        "projectilePrototype", "FireballPrototype")
    local bolt = Engine.SpawnFromObject(prototype, x, y, z)
    if bolt == nil then
        Engine.Log("Fireball prototype was not found")
        return
    end

    local fx, fy, fz = Engine.GetForward()
    local configured = Engine.ConfigureProjectile(
        bolt, fx, fy, fz,
        Engine.GetFieldFloat("projectileSpeed", 18.0),
        Engine.GetFieldFloat("projectileDamage", 30.0),
        30.0, 0.12, Engine.Self())
```

```
    if not configured then
        Engine.Destroy(bolt)
        Engine.Log("Spawned object has no Projectile component")
        return
    end

    local cue = Engine.GetFieldAsset("whooshCue", "")
    if cue ~= "" then Engine.PlayAudioCue(cue, true) end
    Engine.ShakeCamera(0.20, 0.12, 24.0)
end

function OnDestroy()
end
```

Lua does not require the script compiler. Save the file, leave and re-enter Play, then inspect the Console for `[Lua] WizardCombat ready`. `SpawnFromObject` accepts the saved object/prefab name. `ConfigureProjectile` normalizes the direction, assigns speed, damage, range, radius, and owner, and returns `false` when the spawned object lacks a Projectile component. Lua function-name timers resolve global Lua functions directly and do not need a separate binding call.

Completion test

- One click produces one action and one projectile.
- The named cooldown timer becomes active after casting and re-enables casting

only after it completes.

- Projectile starts at StaffTip, not the character origin.
- It travels forward.
- A missed projectile never reduces player/enemy health.
- A wall consumes the projectile before a target behind the wall.

14. M10 - Add game rules, save data, and scene flow

Create an empty `GameManager` and attach `WizardGameManager`.

Responsibilities:

- call `Game()->Reset()` at the beginning of a run;
- count defeated enemies/runes;
- call `Game()->AddScore(...)`;
- choose Explore, Alert, Combat, or Victory music;
- save rune and checkpoint state with `SaveValue`/`SaveCheckpoint`;
- restore with `LoadValue`/`LoadCheckpoint`;
- play the opening and victory camera sequences;
- request another standalone scene with `RequestSceneLoad`, or manually stream a

level with `RequestLevelLoad`/`RequestLevelUnload`;

- apply victory slow motion and short impact hit stops;
- call `Game()->Win(...)` or `Game()->Lose(...)`.

Example control fragment:

CPP

```
void OnCreate() override {
    BindTimerFunction("RestoreNormalTime", [this]() {
        SetGlobalTimeDilation(1.0f);
    });
    if (Game()) Game()->Reset();
    Audio().LoadMusic(
        "Content/Assets/Audio/WizardMusic.music");
    Audio().SetMusicState("Explore", false);
    Camera().PlaySequence("ObservatoryOpening", true, true);
}

void OnUpdate(float) override {
    if (!Game()) return;
    if (WasCameraSequenceEvent(
            "ObservatoryOpening", "ControlBegins")) {
        Audio().SetMusicState("Explore", true);
    }
    if (Game()->IsWon() && !victoryStarted_) {
        victoryStarted_ = true;
        HitStop(0.08f);
        SetGlobalTimeDilation(0.35f);
        SetTimerByFunctionName(
            "RestoreNormalTime", 0.35f, false);
        Audio().SetMusicState("Victory", true);
        Camera().PlaySequence("Victory", true, false);
        SaveValue("ObservatoryComplete", "1");
    }
}

bool victoryStarted_ = false;
```

Use a scripted `Sequence().Do().Wait().WaitUntil()` for multi-step gate or checkpoint logic instead of nesting timer callbacks.

For the manual arena entry, a trigger script can call:

CPP

```
RequestLevelLoad("Arena");
```

After the player leaves and no persistent object references arena entities:

CPP

```
RequestLevelUnload("Arena");
```

Level requests accept the manifest path, file name, or file stem and are consumed after script update. They work only when the startup asset is a `.3dgworld`. Create separate Title and gameplay startup assets if desired;

`RequestSceneLoad` remains appropriate for replacing the entire registry.

Global time dilation scales gameplay scripts, normal timers, physics, AI, animation, particles, and gameplay cameras. The editor UI and audio mixer remain responsive. `HitStop` counts down using unscaled real time, so a zero-dilation freeze always releases. In this example, the timer waits for `0.35` scaled gameplay seconds at `0.35x`, producing about one second of visible slow motion after the 80 ms hit stop.

14.1 Lua game-manager version

Attach this Lua alternative to the empty `GameManager`:

LUA

```
local victorySequenceStarted = false

function OnCreate()
    Engine.SetGlobalTimeDilation(1.0)
    Engine.ResetGame()
    Engine.LoadAdaptiveMusic(
        "Content/Assets/Audio/WizardMusic.music")
    Engine.SetMusicState("Explore", false)
    Engine.PlayCameraSequence("ObservatoryOpening", true, true)

    local completed =
        Engine.LoadValue("ObservatoryComplete", "0")
    Engine.Log("Previous completion: " .. completed)
end

function RestoreNormalTime()
    Engine.SetGlobalTimeDilation(1.0)
end

function OnUpdate(dt)
    if Engine.WasCameraSequenceEvent(
        "ObservatoryOpening", "ControlBegins") then
        Engine.SetMusicState("Explore", true)
    end

    if Engine.GetGameState() == "Won"
        and not victorySequenceStarted then
        victorySequenceStarted = true
        Engine.HitStop(0.08)
        Engine.SetGlobalTimeDilation(0.35)
        Engine.SetTimerByFunctionName(
            "RestoreNormalTime", 0.35, false)
        Engine.SetMusicState("Victory", true)
        Engine.PlayCameraSequence("Victory", true, false)
        Engine.SaveValue("ObservatoryComplete", "1")
    end
end

function LoadArena()
    Engine.RequestLevelLoad("Arena")
end

function UnloadArena()
    Engine.RequestLevelUnload("Arena")
end
```

The active Game Mode owns score, elapsed time, state, and the HUD binding values. Lua reads them with `Engine.GetScore()`, `Engine.GetElapsed()`, and `Engine.GetGameState()`. Use `Engine.GetGlobalTimeDilation()`, `Engine.GetEffectiveTimeDilation()`, and `Engine.IsHitStopActive()` for a slow-motion HUD or debug display. Scene and level requests are deferred until script updates finish.

Completion test

- Score and elapsed time reset on a new run.
- Pause freezes gameplay.
- Player death produces GameOver once.
- Victory state is saved.
- Victory starts with a real-time hit stop, continues in slow motion, and

reliably restores `1.0` global dilation.

- The Arena level loads by manifest name without replacing the persistent player or game manager.
- Restart and scene changes do not leave old scripts or audio sources alive.

15. M11 - Create the enemy character

Duplicate the player character asset as `EnemyWizard.3dgcharacter`, then change:

- team to 2;
- Health to `80-120`;
- player-controller component off;
- navigation agent on;
- enemy collision channel;
- enemy material tint;
- enemy cast and death action clips;
- behavior tree asset assigned from a dropdown;
- scripts: `WizardEnemyCombat`, `WizardLifeReactions`, and optional loot script.

Add Ragdoll:

TEXT

```
Enabled             On
Activate on death   On
Total mass          65
Maximum bodies      16
Death impulse       1.5
```

The authored capsule remains upright while alive. Render-only model orientation does not rotate the collider.

Completion test

- Enemy preview uses its own model and graph.
- Collider guides match the character.
- Vision guides point from its visual front.
- Death preview switches from animation to physics only once.

16. M12 - Bake navigation and grounded AI movement

16.1 Nav Mesh Bounds Volume

Add `NavMeshBoundsVolume_Main` and scale it around all intended walkable ground. The volume is editor-only guidance and does not create collision.

Build navigation. Enable walkable-area debug:

- green: walkable;
- red boundary/debug marks: excluded edge, invalid clearance, or non-walkable

region depending on the active overlay.

Rebuild after changing static colliders or bounds.

16.2 Agent settings

Start with:

TEXT

```
Speed              3.2
Max force          20
Reach radius       2.0
Repath             0.25 s
Vision range       16
Vision half-angle  60 degrees
Ground probe       0.25
Step height        0.35
Maximum slope      50 degrees
Team               2
```

Choose `PlayerWizard` as the chase target or enable nearest hostile targeting with nonzero opposing teams.

16.3 Navigation and steering systems

The navmesh provides polygons and paths. A* finds a route. Steering turns the route into acceleration using Seek, Arrive, FollowPath, Wander, Flee, Pursue, or Evade. Ground probing and gravity keep the agent from floating through floors. Agent collision/crowd response prevents enemies occupying the same space.

Keep the nav-grid fallback available for simple grid-authored areas, but use the navmesh for the observatory.

Completion test

- The bake has nonzero polygons.
- Player, enemies, patrol points, stairs, and arena are on green areas.
- Enemy moves horizontally while gravity/floor probes control height.
- Stairs and slopes do not cause snapping or underground movement.
- Walls block navigation and physical movement.

17. M13 - Build the behavior tree and blackboard

Create:

TEXT

```
Content/Assets/AI/EnemyWizard.btgraph
```

17.1 Blackboard

Add:

Key	Type	Initial value
hasTarget	Bool	false
targetAlive	Bool	true
distanceToTarget	Float	999
castRange	Float	7
retreatRange	Float	2.5
lastSeenPosition	Vec3	0 0 0
alerted	Bool	false
casting	Bool	false

Do not initialize a `TargetDead` key to true. A target without Health should be treated according to the authored rule, not automatically dead.

17.2 Graph

TEXT

```
ROOT Selector "Wizard Combat"
  [Service: WizardSenseService, 0.05 s]
  |
  +-- Sequence "Dead Target"
  |     [Decorator: Check targetAlive == false]
  |     +-- Clear Focus
  |     +-- Patrol
  |
  +-- Sequence "Retreat"
  |     [Decorator: distanceToTarget < retreatRange]
  |     +-- Focus Target
  |     +-- Script Task: RetreatFromTarget
  |
  +-- Sequence "Cast"
  |     [Decorator: hasTarget == true]
  |     [Decorator: targetAlive == true]
  |     [Decorator: Target Within castRange]
  |     [Decorator: Cooldown 1.5 s]
  |     +-- Focus Target
  |     +-- Script Task: WizardCastTask
  |
  +-- Sequence "Chase"
  |     +-- See Target?
  |     +-- Focus Target
  |     +-- Chase [Service: Repath 0.25 s]
  |
  +-- Sequence "Search"
  |     [Decorator: alerted == true]
  |     +-- Move To lastSeenPosition
  |     +-- Wait
  |
  +-- Sequence "Patrol"
        +-- Clear Focus
        +-- Patrol
```

Rename composites for readability. Save must overwrite `EnemyWizard.btgraph`, never append another `.btgraph`.

17.3 Service

Create the service directly from the Behavior Graph's integrated BT-script workflow. Start from the Service template:

CPP

```
class WizardSenseService final : public engine::ai::BtScript {
public:
    engine::ai::BtStatus Tick(
        engine::ai::AgentContext& c, float) override {
        const bool valid = c.registry
            && c.targetEntity != engine::ecs::kNull
            && c.registry->Valid(c.targetEntity);
        bool alive = false;
        float distance = 999.0f;
        if (valid) {
            distance = glm::length(
                c.targetPos - c.agent.position);
            const auto* health =
                c.registry->TryGet<engine::Health>(
                    c.targetEntity);
            alive = !health || health->alive;
            if (c.seesTarget) {
                c.blackboard.SetVec3(
                    "lastSeenPosition", c.targetPos);
                c.blackboard.SetBool("alerted", true);
            }
        }
        c.blackboard.SetBool("hasTarget", valid);
        c.blackboard.SetBool("targetAlive", alive);
        c.blackboard.SetFloat(
            "distanceToTarget", distance);
        return engine::ai::BtStatus::Success;
    }
};
```

17.4 Decorators and tasks

Use:

- bool decorators as checkboxes/boolean values, not float editors;
- Target Within and visibility decorators;
- Cooldown to prevent repeated cast damage;
- a script decorator for any compound condition;
- `WizardCastTask` to play the enemy action and request one projectile at the

release event;

- `RetreatFromTarget` using Flee steering;
- services only for periodic observation/repath, not one-time attacks.

BT scripts use `OnEnter`, `Tick`, and `OnExit`. Return `Running` while an action is in progress, `Success` when complete, and `Failure` when its prerequisites become invalid.

Behavior-tree extension tasks, decorators, and services remain native `engine::ai::BtScript` classes. Lua gameplay scripts can still publish component state, react to animation events, and perform the projectile release selected by the tree.

17.5 Focus and turning

Focus Target updates facing continuously while visible. Clear Focus removes it after sight is lost. Turn speed should be fast enough to track the player without snapping. Stop steering inside reach/cast range; resume only when the player leaves that range.

17.6 State machine usage

Use the engine state machine for a simple `ArcaneSentinel` prop with states:

TEXT

```
Dormant -> Alert -> Firing -> Cooldown -> Dormant
```

The behavior tree remains the main enemy brain. The small state machine demonstrates lightweight state-driven logic for an object that does not need a tree.

Completion test

- Live graph borders show the active branch.
- Blackboard targetAlive follows actual Health.
- Enemy does not remain in the dead-target branch while the player is alive.
- Focus updates immediately when the player changes direction.
- Cast branch stops movement in range and cooldown prevents spam.
- Losing sight eventually clears focus and enters search/patrol.

18. M14 - Enemy spell, damage, death, and ragdoll

Enemy fireballs use a separate prefab/material/particle color and owner team. The cast task or enemy combat script:

1. plays `EnemyStaffCast`;
2. waits for `Cast.Release`;
3. resolves the enemy `StaffTip`;
4. aims toward the current player position;
5. spawns `EnemyFireballPrototype`;
6. sets Projectile owner, direction, speed, damage, radius, and range;
7. plays EnemyCast cue;
8. returns Running until the action completes.

Damage is applied by the projectile hit, not by cast distance or a timer. Therefore a dodge, wall, or missed projectile causes no damage.

On Health `justDied`:

- play death particles/audio once;
- add score;
- clear AI focus and steering;
- disable attack scripts;
- activate Ragdoll;
- optionally spawn a rune pickup;
- destroy or settle the ragdoll after a configured delay.

Ragdoll builds physics bodies and bone drivers from the skeleton. It disables the root collider response needed during life and replaces animation after death.

Add a second script slot named `WizardHitReaction.lua` to each damageable wizard. It turns confirmed health changes into a short impact freeze:

LUA

```lua
local previousHealth = nil

function OnCreate()
    previousHealth = Engine.GetHealth()
end

function OnUpdate(dt)
    local health = Engine.GetHealth()
    if health == nil then return end

    if previousHealth ~= nil and health < previousHealth then
        Engine.HitStop(0.055)
    end
    previousHealth = health
end
```

This observes the authoritative Health component after projectile collision. It does not predict damage from cast distance. Because hit-stop duration uses unscaled time, the 55 ms freeze ends even though gameplay reaches an effective dilation of zero. Avoid starting hit stops from both the projectile and the victim unless the stronger combined effect is intentional.

18.1 Lua enemy projectile release

Attach `WizardEnemyCombat.lua` to the enemy character. Let the behavior tree decide when to cast; the Lua script performs the event-timed release:

LUA

```
function OnUpdate(dt)
    if not Engine.WasAnimationEvent("Cast.Release") then
        return
    end

    local target = Engine.FindObject("PlayerWizard")
    if target == nil then return end

    local sx, sy, sz = Engine.SocketPosition("StaffTip")
    local tx, ty, tz = Engine.GetPosition(target)
    if sx == nil or tx == nil then return end

    local dx, dy, dz = tx - sx, ty - sy, tz - sz
    local length = math.sqrt(dx * dx + dy * dy + dz * dz)
    if length < 0.001 then return end

    local bolt = Engine.SpawnFromObject(
        "EnemyFireballPrototype", sx, sy, sz)
    if bolt == nil then return end

    if not Engine.ConfigureProjectile(
        bolt, dx / length, dy / length, dz / length,
        14.0, 18.0, 28.0, 0.14, Engine.Self()) then
        Engine.Destroy(bolt)
        return
    end

    Engine.PlayAudioCue(
        "Content/Assets/Audio/EnemyCast.3dgaudio", true)
end
```

The BT cast task plays `EnemyStaffCast`; this script observes its authored release event. Keep the Cooldown decorator in the tree so the task cannot start every frame. Projectile collision, not cast distance, decides whether damage occurs.

Completion test

- Enemy and player projectiles ignore their owners.
- Health decreases only on a confirmed collision.
- A confirmed health reduction creates one short hit stop; a miss creates none.
- `justDied` reactions occur once.
- Ragdoll inherits the final pose and does not fall through the floor.
- World health bar disappears for dead enemies.

19. M15 - Create HUD, title, pause, and results screens

19.1 Gameplay HUD

Create:

TEXT

```
Content/Assets/HUD/WizardGameplay.hud
```

Add:

- Panel background;
- player health Bar bound to HealthFraction;
- health Text bound to HealthText;
- score Text: `Score: {}` bound to named float/string supplied by runtime;
- time Text: `Time: {}` bound to `time`;
- game-state Text bound to `gamestate`;
- message Text bound to game message;
- spell icon Image using an engine texture;
- cooldown/progress Bar using a named float if published by project HUD context.

The runtime game-mode bridge supplies score, time, state, and message. Binding keys and source types are case-sensitive.

19.2 Enemy bars

Enable world health bars. They project the collider top to screen space for each living Health actor except the local player.

19.3 Title and pause

Create Title and Pause HUD documents with:

- Play/Resume;
- Restart;
- Options placeholder;
- Quit/Exit Play;
- background Image;
- optional Unlit/UI shader.

Buttons use built-in actions or EmitEvent keys consumed by the runtime host/game logic. Pause applies the Paused audio snapshot and frees the cursor.

19.4 Use in scene

Click **Use in Scene** for the correct gameplay HUD or select it in Game Mode Settings. Save the HUD after editing.

Completion test

- Player health updates immediately.

- Enemy health bars track the correct actors.
- Score, time, state, and message change during Play.
- Buttons work only with an active UI cursor.
- The HUD image and optional UI shader resolve through engine-owned assets.

20. M16 - Cameras, spline sequences, zones, and shake

20.1 Saved cameras

In Camera Manager capture:

TEXT

```
Opening_Start
Opening_Observatory
Arena_Reveal
Victory_Orbit
```

20.2 Opening sequence

Create `ObservatoryOpening`:

- use saved cameras or a spline rail;
- add keyframes for position, rotation, FOV, and duration;
- add event `ControlBegins`;
- lock player input;
- make it skippable;
- blend into the gameplay camera.

Spline math provides smooth camera rail interpolation.

20.3 Camera zones

Add:

- an isometric camera zone over the rune puzzle;
- an indoor zone with shorter distance;
- an arena trigger that starts `ArenaReveal`.

The gameplay camera mode is authored before Play. Zones and sequences override it deliberately; player input does not cycle global modes.

20.4 Shake

Use mild translation/rotation shake:

- staff cast: small/short;
- impact near camera: distance-scaled;
- gate opening: low-frequency;
- boss death: stronger, brief, with optional FOV amplitude.

Completion test

- Sequence rails appear only when their debug toggle is enabled.
- Input locks and restores correctly.
- Skip emits the expected result.
- Zone entry/exit blends instead of snapping.
- Camera collision remains active during ordinary gameplay.

21. M17 - Add no-code and component-driven gameplay

Use component systems to reduce script code:

- Rotator on rune pickups and floating crystals;
- Mover on a platform or gate;
- TriggerAudioAction for the indoor ambience transition;
- AudioSource for looping water, wind, and portal sounds;
- ParticleSystemComponent for rune idle effects;
- CameraZone for view changes;
- Health on damageable props;
- Empty objects for managers, audio emitters, triggers, and effect prototypes.

Build a rope bridge or suspended lantern using distance/rope/spring joints. Add dynamic bodies only where interaction is worth the physics cost.

Completion test

- Each component works in editor Play and exported runtime.
- Empty objects have line icons/guides, not visible cube geometry.
- No component system is duplicated by a per-entity world loop script.

22. M18 - Debug, validate, and optimize

22.1 Editor panels to use

Panel	Validation task
Viewport	composition, selection, gizmos, drag/drop
Hierarchy	names, object count, selection
Inspector	component data and script fields
Assets	registry type, paths, imports, double-click routing
World Settings	sky, clouds, lighting, shadows, post-process
World Editor	persistent level, streamed levels, transforms, rules, radii, and world cook
Game Mode Settings	player, HUD, startup scene, fixed camera policy
Material Maker	PBR maps and shader selection
Shader Editor	compile and graph errors
Character Editor	model, collider, graph, sockets, scripts, AI
Graph Editor	locomotion, blend space, transitions
Clip Editor	action clips and events
Behavior Graph	live branch and blackboard
Particle Editor	module stack and preview
Audio Editor/Mixer	waveform, cues, buses, snapshots, music
HUD Editor	widget bindings and actions
Camera Manager	saved cameras, shake, sequences
Console	load, validation, compile, runtime errors
Profiler	CPU/GPU frame cost
Physics Status	bodies and contacts
Gameplay Debug	AI, navigation, projectiles, health
Script API/Debug	factories, attachment, lifecycle, errors

22.2 Runtime update order

Preserve this dependency order:

TEXT

```
Unscaled clock and hit-stop countdown
Global time-dilation calculation
Input and variable script update using scaled delta
Game mode/rules
Fixed script update
Player and gameplay systems
AI, behavior tree, steering, grounded movement
Physics simulation and collision events
Projectile hits and health
Animation pose/action update
Ragdoll activation/update
Attachments and sockets
Audio source/listener update
CPU/GPU particle simulation
Shadow passes and scene rendering
Post-processing
HUD and world health bars
Optional debug/editor overlays
Level-streaming decision around the current viewer
```

The host may group operations differently, but dependencies must remain valid: damage follows collision, sockets follow current animation, and HUD follows gameplay state. Hit-stop must advance from unscaled time before the

scaled gameplay delta is calculated. Streamed-level activation must apply its placement and resolve assets before building physics, animation, AI, audio, particles, or scripts for the newly created entities.

22.3 Performance pass

Measure before changing quality:

- hide navigation, collider, camera, and AI debug overlays;
- reduce excessive shadowed point/spot lights;
- limit shadow visibility by quality target, not by accidental short defaults;
- use frustum culling;
- cap particle counts and cue voices;
- prefer GPU particles for large effects, CPU for portability/debug;
- reduce distant grass and particle density, and keep grass height variation

restrained so tall blades do not increase unnecessary overdraw;

- inspect render scale, SSR, SSAO, and shadow cost separately;
- confirm editor UI cost is not being mistaken for runtime rendering cost.

22.4 Automated tests

Run the test suite:

POWERSHELL

```
cmake --build build --config Debug
ctest --test-dir build -C Debug --output-on-failure
```

Focused targets cover particles, animation movement, cameras, audio, shaders, materials, scripts, runtime AI, image decoding, editor assets, registry, cooking, static meshes, skeletal meshes, named timers, global gameplay time, Lua integration, world manifests, and level streaming.

Completion test

- F8 validation reports no missing dependency, script factory, malformed asset,

zero-polygon navmesh, or shader compile failure.

- No Dear ImGui conflicting-ID warning appears.
- Debug overlays are off for final gameplay.
- A repeatable profiler capture meets the project frame target.
- Streaming boundaries do not cause duplicate runtime systems or a visible

multi-level activation hitch.

23. M19 - Export, cook, and package

23.1 Save and validate

Save all authored assets and the scene. Validation checks references and serialization before packaging.

23.2 Export runtime scene

F7 exports the editor scene into player-readable runtime data. Editor-only selection, preview, and debug state are removed.

For the final game, open World Editor and **Cook World**. This exports the persistent and streamed editor scenes, calculates their bounds, preserves placement transforms and streaming rules, and writes the cooked `WizardsTrialWorld.3dgworld` startup asset.

23.3 Cook assets

The Asset Cooker:

- starts from the runtime scene or world manifest;
- follows registry dependencies;
- copies/transforms reachable payloads;
- excludes editor-only data;
- reports missing dependencies;
- produces runtime registry metadata.
- copies reachable `Content/Scripts/*.lua` files while preserving their

Content-relative paths.

- includes every level referenced by the cooked world and the native assets

reachable from those levels.

The packaged player must not require the original FBX, Gaea project, texture source directory, or editor.

23.4 Build player

POWERSHELL

```
cmake -S . -B build `
  -DPLAYER_GAME_DIR="D:/Games/WizardsTrial" `
  -DGAMEENGINE_STATIC_RUNTIME=ON
cmake --build build --config Release --target player
cmake --install build --config Release `
  --prefix dist --component player
```

Launch the executable from `dist` and test:

- `.3dgworld` startup and persistent level;
- distance and manual streamed-level activation/unload;
- input and camera;
- scripts;
- animation/action events;
- navigation and behavior trees;
- particles and audio;
- HUD/buttons;

- scene transitions and save data;
- victory and restart.

Completion test

Disconnect or move the external source-asset directory and launch again. The game must run entirely from its cooked project content.

24. Final playtest route

Perform this exact route on a clean run:

1. Start from the title HUD.
2. Watch or skip the opening camera sequence.
3. Walk, run, turn, jump, and climb stairs.
4. Enter the isometric puzzle zone.
5. Collect the rune without blocking the capsule or camera.
6. Activate the physics gate and hear its spatial cue.
7. Move between outdoor and indoor audio snapshots.
8. Approach an enemy from outside vision and watch patrol.
9. Enter vision and verify focus/chase.
10. Move inside cast range and verify stop/cast/cooldown.
11. Dodge a projectile and confirm no damage.
12. Let a projectile hit and confirm health/HUD/shake.
13. Cast while moving with the masked action.
14. Cast a full-body action and confirm movement lock.
15. Kill the enemy and verify score, particles, audio, health bar removal, and ragdoll.
1. Trigger arena reveal and combat music.
2. Defeat the guardian and verify victory sequence/save data.
3. Restart and verify a clean runtime state.
4. Repeat the route in the standalone player.

25. Troubleshooting

An asset is black or unchanged

- Confirm the material was saved and assigned to the correct slot.
- Confirm native texture dependencies exist in the registry.
- Confirm texture color space and decode succeeded.
- Confirm the material/shader domain is correct.
- Confirm lights and exposure are present.

The character faces down or backward

- Correct model orientation in Character Editor's render-only transform.
- Do not rotate the scene collider to fix imported mesh axes.
- Verify gameplay forward and the vision debug arrow.

The fireball does not appear

- Confirm the prototype name/asset field.
- Confirm `Cast.Release` exists and fires.
- Confirm StaffTip resolves.
- Confirm the projectile/particle component is enabled.
- Confirm spawn position is in front of the near plane and outside the owner collider.

The fireball travels backward

- Derive direction from the character's gameplay forward.
- Do not use the render-only model correction as world rotation.
- Inspect the StaffTip forward guide.

Enemy stands still

- Confirm behavior tree dropdown assignment.
- Confirm graph validates and its root is set.
- Confirm navmesh polygon count is nonzero.
- Confirm both actors are on walkable coverage.
- Confirm target name/team.
- Confirm BT script factories are registered.
- Inspect live blackboard and node borders.

Enemy clips into ground or stairs

- Check visible model offset versus capsule.
- Check ground probe, gravity, step height, and maximum slope.
- Rebuild navmesh after floor/collider edits.

- Confirm AI steering does not write vertical position directly.

HUD values remain placeholders

- Use the correct active HUD.
- Match case-sensitive binding keys and source types.
- Confirm GameMode updates during Play.
- Confirm the script is attached, enabled, compiled, and running.

A Lua script does not run

- Confirm the file ends in `.lua` and is under the active project's `Content/Scripts` folder.
- Select it from the saved-script dropdown instead of entering an external path.
- Confirm **Script Enabled** is checked on that script slot.
- Match lifecycle names exactly: `OnCreate`, `OnUpdate`, `OnFixedUpdate`, and `OnDestroy`.
- Leave and re-enter Play after saving so the Lua state reloads.
- Read the Console error, which includes the script path and Lua line number.
- Confirm called `Engine.*` functions exist in the current engine build.
- Confirm the cooker copied the Lua file for the standalone player.

A named timer never calls the function

- In C++, call `BindTimerFunction` before `SetTimerByFunctionName`.
- In Lua, define the named function in the global script scope.
- Check the returned timer handle or `IsTimerActive` result.
- Remember that normal timers consume scaled gameplay time. At global dilation `0.0`, they pause until gameplay resumes.
- Use `ClearTimerByFunctionName` when replacing or cancelling a repeating timer.

Slow motion or hit stop never returns to normal

- Bind or define the function that restores global dilation to `1.0`.
- Remember that the recovery timer is also scaled; duration divided by dilation approximates its real-time length.
- Do not use a normal zero-dilation timer to recover from a complete freeze. `HitStop` already owns an unscaled recovery clock.
- Reset global dilation in game-manager `OnCreate` and during restart.

A streamed level does not appear

- Start the player with the cooked `.3dgworld`, not one child scene.
- Confirm the world has a valid persistent scene and the child scene path is present in its manifest.

- For Distance rules, check world placement, calculated bounds, and load radius.
- For Manual rules, call `RequestLevelLoad` or `Engine.RequestLevelLoad` using the manifest path, file name, or stem.
- Read the Console streaming warning for a load, instantiate, or dependency error.

Dear ImGui reports conflicting IDs

- Give repeated controls unique `##suffix` identifiers.
- Push an object/node ID around loop-generated widgets.
- Do not use empty labels without a unique hidden ID.

26. Every-system signoff

Use this checklist before calling the tutorial complete.

Core, ECS, and scene

- [ ] Application loop runs variable presentation and fixed simulation.
- [ ] Window input, cursor capture, aspect, VSync, and resize work.
- [ ] ECS registry creates, queries, clones, and destroys entities safely.
- [ ] Components serialize into editor and runtime scenes.
- [ ] Empty objects and prefabs work.
- [ ] Last saved scene restores.
- [ ] A `.3dgworld` keeps its persistent level resident and correctly loads, places, and unloads Distance, Always Loaded, and Manual level assets.
- [ ] Streamed-level scripts and transient physics, AI, audio, animation, and particle state initialize and shut down exactly once.

Assets

- [ ] Stable handles and registry dependencies resolve.
- [ ] Static mesh, skeletal mesh, animation, texture, material, shader, particle, audio, HUD, character, graph, clip, behavior, prefab, and scene assets open.
- [ ] Import, destination selection, rename, reimport, and cooking work.
- [ ] Runtime manager caches resolved assets.

Rendering

- [ ] Basic/static and PBR meshes render.
- [ ] Skinned renderer uses current animation pose.
- [ ] Directional, point, spot, and area lights work.
- [ ] Cascaded, point, and spot shadows work at authored distances.
- [ ] IBL, procedural sky, day/night, clouds, and cloud shadows work.
- [ ] Terrain, toggleable grass-height randomization, spline-driven water, culling, and spline cameras work.
- [ ] Bloom/post-process, SSAO, SSR, anti-aliasing, and render scale work.
- [ ] GPU profiler identifies scene and UI costs.

Materials and shaders

- [ ] Every PBR channel used by the project loads correctly.
- [ ] Material instance/scene overrides work.
- [ ] Surface, Unlit/UI, and Post-process shader domains compile.
- [ ] Shader parameters and engine textures resolve in editor and player.

Animation

- [ ] Skeleton, model, clips, animator, and controller work.
- [ ] Graph parameters and multi-condition transitions work.
- [ ] Blend space is smooth and synchronized.
- [ ] Root motion is deliberate.
- [ ] Action clips, masks, movement lock, events, sockets, and attachments work.

Physics

- [ ] All used collider shapes match their guides.
- [ ] Rigid bodies, solver, contacts, events, queries, and channels work.
- [ ] Trigger/ignore/block responses are correct.
- [ ] Joints and character controller are stable.
- [ ] AI grounding and ragdoll work.

AI

- [ ] Nav bounds, navmesh, optional grid, A*, and paths work.
- [ ] Seek/Arrive/FollowPath/Flee steering is demonstrated.
- [ ] Perception, vision direction, teams, focus, and clearing focus work.
- [ ] State machine example changes state.
- [ ] Behavior graph, blackboard, built-in nodes, BT tasks, decorators, and services all run.

Gameplay and scripting

- [ ] Game mode state, score, timer, victory, defeat, pause, and message work.
- [ ] Player controller and camera policy work.
- [ ] Health, projectile, attachment, rotator, mover, and camera zone work.
- [ ] Multiple scripts attach and lifecycle methods execute.
- [ ] Native C++ and Lua scripts both run in Editor Play and the player.
- [ ] Lua fields are detected and editable in the Inspector/Character Editor.
- [ ] Lua projectile spawning uses a valid prefab, socket, direction, and owner.
- [ ] Event timers and named-function timers can be queried, cancelled, repeated, and used from both C++ and Lua.
- [ ] Global time dilation affects gameplay, physics, AI, animation, particles, cameras, and normal timers while editor UI and audio mixing remain responsive.
- [ ] Hit stops use unscaled duration and recover from an effective zero-dilation freeze.
- [ ] C++ and Lua scripts can request manual streamed-level load and unload.
- [ ] C++ and Lua can read, move, rotate, add, insert, remove, translate, close, and sample spline points; connected rivers update during Play.

- [ ] Input, fields, timers, sequence, collision events, animation events,

persistence, scene load, particles, audio, cameras, and animation script APIs are exercised.

Audio, particles, and UI

- [ ] Fire-and-forget and persistent spatial sources work.
- [ ] Cues, buses, effects, snapshots, adaptive music, attenuation, doppler,

cones, occlusion, and priority are configured.

- [ ] CPU and GPU particle paths, all module stages, local/world space, bursts,

trails, and scripting work.

- [ ] HUD Panel, Text, Bar, Button, and Image widgets work.
- [ ] Anchors, bindings, button actions, fonts, and world health bars work.

Editor, build, and tests

- [ ] Content double-click routes every asset to its specialist panel.
- [ ] Inspector hides irrelevant type-specific settings.
- [ ] Gizmos, line guides, panels, console, profiler, and debug toggles work.
- [ ] Script creation, compilation, external editor selection, and errors work.
- [ ] Runtime export, validation, cooking, player build, and tests pass.
- [ ] World cook packages the persistent scene, every referenced streamed level,

Lua scripts, and all reachable native dependencies.

When every box is checked, Wizard's Trial is not only a playable tutorial game: it is an integration test for the complete 3DGEngine toolchain.